



DOCKER NOTES



Build once,
Run anywhere
- Docker

1. WHAT IS DOCKER?

Docker is a platform that helps us **develop**, **ship**, and **run** applications inside **containers**. It packages an application and all its dependencies together so it can run **consistently** on any environment.

CONTAINERS vs VIRTUAL MACHINES

	Containers	Virtual Machines
Architecture	Share OS Kernel	Have their own OS
Size	Small (MBs)	Large (GBs)
Boot Time	Seconds	Minutes
Performance	Near Native	Lower
Isolation	Process-level	Full Machine-level
Use Case	Microservices, Dev, CI/CD	Legacy apps, Different OS

WHY DO WE USE DOCKER?

- Lightweight compared to VMs
- Consistency across environments
- **Isolation** (separating applications safely)
- Portability (move applications easily)
- Better resource utilization
- Faster deployment
- **Scalability** (ability to handle growth)

CONTAINER



VIRTUAL MACHINE

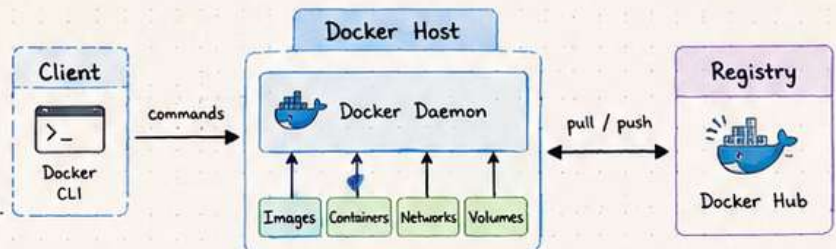


→ Containers share the host OS kernel, so they are much more **efficient**!

2. DOCKER ARCHITECTURE

Docker follows a **Client-Server** architecture.

- Docker Client (CLI) → You run commands.
- Docker Daemon (Server) → Does all the heavy lifting.
- Docker Objects → Images, Containers, Networks, Volumes.
- Docker Registry → Stores Docker Images (e.g., Docker Hub).



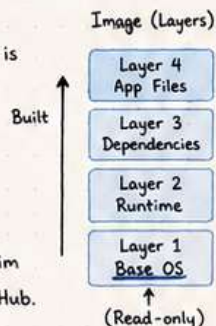
3. DOCKER IMAGE

An image is a read-only template used to create containers.

- Built from a Dockerfile.
- Made of layers (each layer is read-only).
- Images are **immutable** (cannot be changed).
- Stored in a registry.

Example:

nginx:latest, python:3.11-slim
These are images on Docker Hub.



4. CONTAINER

A container is a running instance of an image.

- It has its own filesystem, network, process space.
- Ephemeral (can be removed easily).
- Multiple containers can run from the same image.

Example:

```
docker run -d -p 80:80 nginx
```

run in detached mode publish port host:container image to use



Quick Tip

Think of image as a class and container as an object. Image is the blueprint, container is the running application.

5. DOCKER REGISTRY

A registry stores Docker images.

- Docker Hub → Official public registry.
- Private registry → For companies.
- We can **PULL** images to use and **PUSH** our own images.



REMEMBER

Image → Build
Container → Run
Registry → Store/Share

COMMON MISTAKE

Trying to edit data inside the container thinking it's permanent. Containers are **ephemeral**! Use volumes for persistent data.



DOCKER NOTES

Build • Ship • Run Anywhere

1) DOCKERFILE INSTRUCTIONS

FROM

Sets the base image for the build.
Example: `FROM python:3.11-slim`

Every Dockerfile must start with FROM

RUN

Executes commands while building the image.
Creates a new layer.
Example: `RUN apt-get update && apt-get install -y curl`

COPY

Copies files/folders from host to image.
Example: `COPY . /app`

ADD

Like COPY but can also download files from URL and auto-extract tar files.
Example: `ADD https://example.com/app.tar.gz /app/`

CMD

Default command to run container.
Can be overridden.
Example: `CMD ["python", "app.py"]`

ENTRYPOINT

Configures a container that will run as an executable. Harder to override.
Example: `ENTRYPOINT ["python", "app.py"]`

EXPOSE

Informs Docker that container listens on a port (doesn't publish it).
Example: `EXPOSE 8080`

ENV

Sets environment variables.
Example: `ENV APP_ENV=production`

ARG

Build-time variables (not available at runtime).
Example: `ARG VERSION=1.0`

WORKDIR

Sets working directory for subsequent commands.
Example: `WORKDIR /app`

USER

Sets username or UID to run the container.
Example: `USER appuser`

2' CMD vs ENTRYPOINT

Feature	CMD	ENTRYPOINT
Purpose	Provides default command/args.	Sets the main command (executable).
Override	Easily overridden by <code>docker run ...</code>	Harder to override.
When to use	When you want to provide defaults.	When you want a fixed executable.
Docker run example	<code>docker run image python app.py</code> (uses our CMD)	<code>docker run image arg1 arg2</code> (passed as arguments)

Remember:

If both are used, ENTRYPOINT is the executable and CMD provides default arguments.

Quick Example

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
ENTRYPOINT ["python"]
CMD ["app.py"]
```

`docker run img` → runs: python app.py
`docker run img hello.py` → runs: python hello.py
`docker run -it img bash` → runs: bash
 (ENTRYPOINT is overridden)

Quick Tip

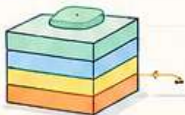
Use CMD for defaults, ENTRYPOINT for core executable.

Common Mistake

Using RUN for things that change often (e.g., copying code) early in the Dockerfile breaks cache and makes builds slower.

3) LAYER CACHING (and how to optimize)

Docker builds images in layers.
Each instruction creates a layer.
Docker reuses (caches) layers when possible.



Dockerfile (Example)	Result
<code>FROM python:3.11-slim</code>	Layer 1 ✓
<code>RUN apt-get update</code>	Layer 2 ✓
<code>RUN pip install -r req.txt</code>	Layer 3 ✓
<code>COPY . /app</code>	Layer 4 ✗
<code>CMD ["python", "app.py"]</code>	Layer 5 ✓

➤ Ideal Order in Dockerfile



Best Practices to maximize cache

- Put stable layers first (base image, system deps).
- Copy dependency files first, install deps, then copy the rest of the code.
- Avoid unnecessary file changes (.dockerignore).
- Use specific base image tags (not latest).
- Combine RUN commands with && to reduce layers.

Memory Trick

"Stable first, volatile last" = Faster builds!

Why?

So that changes in code don't invalidate dependency layers.

4) MULTI-STAGE BUILDS (Smaller images)

Use multiple FROM statements in one Dockerfile.
Copy only what you need to the final image.

Example: Node.js App

```

# Stage 1: Build
FROM node:18 AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Production
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
  
```

How it works

Stage 1 (Build)
Has all build tools + dev dependencies

COPY only /app/dist

Stage 2 (Final)
Only runtime files (nginx + app files)

Result: Much smaller image!

Benefits

- ✓ Smaller image size
- ✓ Faster pull & deploy
- ✓ More secure (no dev tools in final image)

Pro Tip

Name your stages with AS <name> and copy from that name.

5' .DOCKERIGNORE

Works like .gitignore.

Tells Docker what NOT to send to the build context.
Smaller context = Faster builds!

Example .dockerignore

```

node_modules → dependencies
.git → git directory
.env → environment files
*.log → logs
.DS_Store → macOS files
dist/ → build output
build/coverage/ → build output
*.md → test coverage
*.md → markdown files
Dockerfile* → other dockerfiles
  
```

Remember

Only send what's needed to build your image.

Mini Summary

- Dockerfile = blueprint for your image.
- Use layers & caching smartly for faster builds.
- Multi-stage builds = smaller, cleaner images.
- .dockerignore keeps builds efficient.



DOCKER NOTES

3. BASIC DOCKER COMMANDS

Docker has a rich **CLI (Command Line Interface)** that helps us build, run, manage and share containers easily.

QUICK TIP

Most Docker commands follow this structure:

`docker <command> <options> <arguments>`

Docker makes apps portable, consistent and isolated!

REMEMBER

Images are like **blueprints**.
Containers are running **instances** of images.

DOCKER FLOW

Write Dockerfile

Build Image

Run Container

Manage & Monitor



Build once,
Run anywhere!

COMMON MISTAKE

Forgetting to expose ports or mount volumes.
Always check **logs**!

- 1 **docker build** - Build an image from a Dockerfile.

```
docker build -t myapp:1.0 .
```

tag (name:version)

context (current dir)



Creates an image from the instructions in Dockerfile.

Example: Build image named myapp with tag 1.0

- 2 **docker run** - Run a container from an image.

```
docker run -d -p 8080:80 --name web nginx
```

detached (background)

port mapping (host:container)

name for container

image name



Creates and starts a container.

Example: Runs nginx in detached mode and maps port 8080 on host to 80 in container.

- 3 **docker pull** - Download an image from Docker Hub.

```
docker pull ubuntu:22.04
```



Downloads the specified image.

Example: Pull Ubuntu 22.04 image.

- 4 **docker push** - Push an image to Docker Hub.

```
docker push mydockerhubuser/myapp:1.0
```



Uploads your image to a registry.

Example: Push image to your Docker Hub repository.

- 5 **docker ps** - List running containers.

```
docker ps
```

→ To see all (including stopped):

```
docker ps -a
```



Shows currently running containers.

- 6 **docker images** - List local images.

```
docker images
```



Shows all downloaded images.

- 7 **docker logs** - View logs of a container.

```
docker logs -f mycontainer
```



-f follows the log output in real-time.

- 8 **docker exec** - Run a command inside a running container.

```
docker exec -it mycontainer bash
```

interactive

open bash shell



Useful for debugging inside containers.

- 9 **docker stop** - Stop a running container.

```
docker stop mycontainer
```



Gracefully stops the container.

- 10 **docker rm** - Remove a stopped container.

```
docker rm mycontainer
```



Deletes the container.

- 11 **docker rmi** - Remove an image.

```
docker rmi myimage:1.0
```



Deletes the specified image.

- 12 **docker inspect** - Get detailed info about an object.

```
docker inspect mycontainer
```



Shows low-level details in JSON format.

- 13 **docker stats** - Monitor container resource usage.

```
docker stats
```



Shows CPU, memory, network usage in real-time.

- 14 **docker top** - View running processes in a container.

```
docker top mycontainer
```



Shows the processes running inside the container.

- 15 **docker cp** - Copy files/folders between container and host.

```
docker cp mycontainer:/app/file.txt ./
```

from container

to current directory



Copy files from container to host or vice versa.

- 16 **docker commit** - Create a new image from a container's changes.

```
docker commit mycontainer myimage:2.0
```



Save the current state of a container as a new image.

CHEAT SHEET

build → Create image
run → Run container
pull → Download image
push → Upload image
ps → List containers
images → List images
logs → View logs
exec → Run command inside container
stop → Stop container
rm → Remove container
rmi → Remove image
inspect → Inspect details
stats → Resource usage
top → Running processes
cp → Copy files
commit → Save changes as image

KEY TAKEAWAY

- ✓ Images are immutable (unchangeable).
- ✓ Containers are ephemeral (can be removed anytime).
- ✓ Use Docker commands to build, run, ship and scale applications efficiently!



Happy Dockering! 😊

Let's Containerize the world!

DOCKER NOTES

NETWORKING

Docker networking allows containers to **communicate** with each other, with the host, and with the outside world in a **secure** and **efficient** way.

WHY IS NETWORKING IMPORTANT?

- Enables communication between containers and external services (DB, APIs, etc.)
- Provides **isolation** (separating applications safely)
- Allows **scalability** (ability to handle growth)
- Makes deployment **portable** and consistent

Quick Tip

Think of Docker networks like roads. Containers are cars. The network decides how they can reach each other or the outside world!

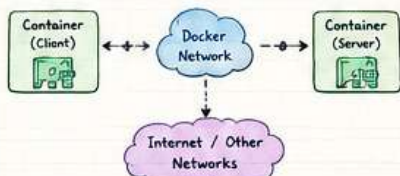
OVERVIEW OF TOPICS

- ✓ Types of Networks: bridge, host, none, overlay
- ✓ Port mapping (-p)
- ✓ Docker DNS & container name resolution
- ✓ Create Custom Network & connect containers

Remember

Containers in the same network can talk to each other using their **CONTAINER NAME** (as long as DNS works).

NETWORK FLOW (BIG PICTURE)



1 TYPES OF NETWORKS

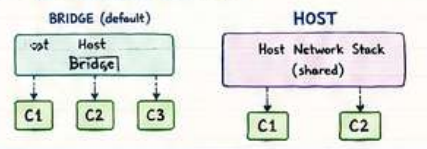
Default network is bridge

Type	Best For	How it Works	Isolation	Use Case
bridge (Default) 	Most applications	Containers connect to a virtual bridge on the host.	Good (Containers isolated from each other by default)	Web apps, APIs, general development
host 	Performance critical apps	Container shares the host's network stack directly.	Low (No isolation)	High performance apps, monitoring tools
none 	Maximum isolation	Container has no network connection.	Very High (No network access)	Highly secure apps, custom network setups
overlay 	Multi-host communication (Swarm)	Connects containers across multiple Docker hosts.	Good	Docker Swarm, Microservices across servers

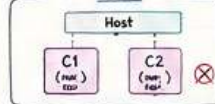
Remember

Bridge is default & works on single host.
Overlay is for multi-host (Docker Swarm).

COMPARISON DIAGRAM



NONE



OVERLAY



Quick Revision

Bridge → single host (default)
Host → shares host network
None → no network
Overlay → multi host

Interview Q

Q: Which network type would you use for high performance monitoring tools? Why?
A: Host, because it avoids network NAT/overhead.

2 PORT MAPPING (-p)

Port mapping allows containers to access services from **outside** the **host machine**.

SYNTAX

`-p <host_port>:<container_port>`

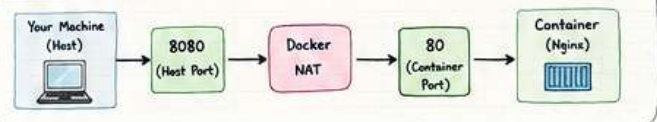
↑ ↑
Port on your machine Port inside the container

EXAMPLE

`docker run -d -p 8080:80 nginx`

↑ ↑
Access Nginx on port 8080 on your browser Nginx runs on port 80 inside the container

HOW IT WORKS



MORE EXAMPLES

- Map 3000 to 3000
`docker run -p 3000:3000 myapp`
- Map multiple ports
`docker run -p 8080:80 -p 443:443 nginx`
- Bind to a specific IP
`docker run -p 127.0.0.1:8080:80 nginx` (only localhost can access)

Common Mistake

Forgetting `-p` means the container port is **NOT** accessible from outside!

Quick Tip

If your app is not accessible, check:

1. Did you use `-p`?
2. Is the app listening on 0.0.0.0 (not localhost)?
3. Firewall?

Real World Example

You run a Blog app in a container (inside port 5000). You want to open it on your browser:

`docker run -p 8080:5000 blog-app`

Now open → `http://localhost:8080`

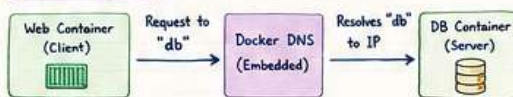
3 DOCKER DNS & NAME RESOLUTION

(Containers talk using names!)

WHAT IS DOCKER DNS?

Docker has an **internal DNS server** (built-in) that resolves container names to IP addresses within the same network.

HOW IT WORKS



Now Client can connect to DB using name "db"

EXAMPLE

`docker run --name db -d postgres`
`docker run --name web -d --network mynet myapp`
• Inside web container, connect using: `db:5432` (Docker DNS resolves "db" automatically)

Remember

Use container names, not IPs. Docker handles IP changes automatically!

Common Mistake

Trying to connect using IP address of another container. IPs can change!

4 CUSTOM NETWORK & CONNECT CONTAINERS

Custom networks give you better control, isolation and allow containers to communicate securely.

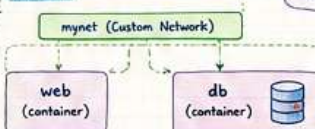
STEPS

1. Create a custom network
2. Run containers and attach to it
3. Containers can talk using names

COMMANDS

- # 1. Create network
`docker network create mynet`
- # 2. Run containers (attach to network)
`docker run -d --name db --network mynet postgres`
`docker run -d --name web --network mynet myapp`
- # 3. Check network & containers
`docker network ls`
`docker network inspect mynet`

VISUAL



Quick Summary

- ✓ Use custom networks for better control
- ✓ Containers talk using names
- ✓ Improves security & isolation
- ✓ Great for real-world apps!

Data is precious! Let's keep it safe with Docker Volumes

DOCKER NOTES

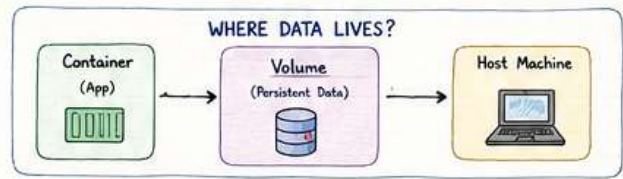


5. VOLUMES & DATA

KEY IDEA

Store data in **VOLUMES** (outside the container) so even if the container is removed, the data stays safe.

Containers are **EPHEMERAL** (temporary). When a container dies, its writable layer is lost. Volumes help us persist and share data safely.



1 VOLUMES vs BIND MOUNTS vs TMPFS

Feature	VOLUMES (Managed by Docker)	BIND MOUNTS (Host Path)	TMPFS (In-Memory)
What is it?	Stored in Docker's managed area on the host.	Map a file or directory on the HOST to the container.	Stored in the HOST memory (RAM), not on disk.
Location	/var/lib/docker/volumes/	Any path on the host system	Inside host memory only
Performance	Good	Good (depends on host disk)	Very Fast (in-memory)
Persistence	Yes (data survives container removal)	Yes (as long as host path exists)	No (data lost on container stop)
Portability	High (easy to move between environments)	Low (depends on host paths)	N/A (temporary)
Use Cases	Databases, logs, backups, production data.	Dev environments, config files, code sharing.	Caching, session data, temp files, sensitive data.

Quick Tip

- Use **VOLUMES** for production data (safer & portable).
- Use **BIND MOUNTS** for development (easy to edit files on host).
- Use **TMPFS** for temporary data that doesn't need to survive.

Remember

Volumes live outside the container lifecycle. Container dies, **DATA SURVIVES!**

2 CREATING & MANAGING VOLUMES

Create a Volume

```
docker volume create mydata
# creates an empty volume
```

Remove a Volume

```
docker volume rm mydata
# must not be in use
```

List Volumes

```
docker volume ls
```

Inspect Volume

```
docker volume inspect mydata
```

Remove Unused Volumes

```
docker volume prune
# removes dangling volumes
```

Use Volume in a Container

-v or --mount

```
docker run -d --name myapp \
-v mydata:/app/data \
nginx
```

mydata = volume name
/app/data = mount point inside container

Check Volume Usage

```
docker system df -v
# shows space used by volumes
```



Common Mistake

Trying to remove a volume that is still used by a container.

Solution:

Stop the container first then remove the volume.

3 DATA PERSISTENCE (KEEP DATA SAFE)

To keep data safe even if a container is removed or recreated.

Example Scenario

You run a database in a container. If you store DB files inside the container, and the container is removed → **DATA LOST!**

Use a volume → **DATA SAFE!**

How Persistence Works



Run DB with Persistent Volume

```
docker run -d --name mydb \
-v dbdata:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=1234 \
mysql:8
```

dbdata = volume name
/var/lib/mysql = where MySQL stores data

Remember

Always mount your important data to a volume!

4 VOLUME SHARING BETWEEN CONTAINERS

Share the same volume between multiple containers.

Example: Two containers sharing the same data volume



Run two containers using the same volume

```
# Writer container
docker run -d --name writer \
-v shared-data:/data \
alpine sh -c "echo Hello > /data/hello.txt && sleep infinity"

# Reader container
docker run -it --name reader \
-v shared-data:/data \
alpine cat /data/hello.txt
```

Output: Hello

They share the same volume!

VOLUME COMMANDS - CHEAT SHEET

Action	Command	Example
Create Volume	docker volume create <name>	docker volume create mydata
List Volumes	docker volume ls	docker volume ls
Inspect Volume	docker volume inspect <name>	docker volume inspect mydata
Use in Container	-v <volume>:/path	-v mydata:/app/data
Remove Volume	docker volume rm <name>	docker volume rm mydata
Remove Unused	docker volume prune	docker volume prune

SUMMARY

- ✓ Volumes keep data safe beyond the container lifecycle.
- ✓ Volumes are managed by Docker and are portable.
- ✓ Bind Mounts map host paths (great for dev).
- ✓ tmpfs stores data in memory (super fast but temporary).
- ✓ Share volumes between containers for collaboration.

Golden Rule

If your app creates data, store it in a **VOLUME!**

Future You will thank you!

Think of **VOLUMES** as a **SAFE BOX** that stays even if the container disappears!

DOCKER NOTES

6. DOCKER COMPOSE

Docker Compose helps us run multi-container applications using a single YAML file.

What is Docker Compose? ♥

It is a tool for defining and running multi-container Docker applications using a YAML file (docker-compose.yml).



1 docker-compose.yml Structure

```
1 version: '3.8'           → Compose file version
2 services:                 → Service (a container)
3   web:                    → Port mapping
4     build: .               → Port mapping
5     ports:                 → Environment variables
6       - "5000:80"         → ASPNETCORE_ENVIRONMENT=Production
7     environment:          → Start order
8       - db
9     depends_on:           → Connect to network
10      - mynet
11   db:
12     image: mysql:8         → Use an image
13     environment:
14       - MYSQL_ROOT_PASSWORD=1234
15     volumes:
16       - db_data:/var/lib/mysql → Persist data
17   networks:
18     - mynet
19 volumes:                  → Define volumes
20   db_data:
21 networks:                 → Define networks
22   mynet:
23 driver: bridge
```

2 services, networks, volumes in Compose

services

- The heart of Compose.
- Each service = one container.



networks

- Connect services to communicate with each other.



volumes

- Store data outside containers (Persistent).



Quick Tip

Compose creates a default network for all services automatically!

Why use them?

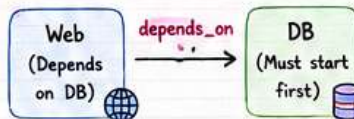
- Services → Run containers
- Networks → Let containers talk to each other
- Volumes → Keep data safe

3 Main Commands

<code>docker compose up</code>	Start all services (create + run)
<code>docker compose up -d</code>	Run in detached mode (background)
<code>docker compose down</code>	Stop & remove containers, networks
<code>docker compose build</code>	Build or rebuild images
<code>docker compose logs</code>	Show logs from services
<code>docker compose exec <service> <cmd></code>	Run a command in a service.
<code>docker compose ps</code>	List running services
<code>docker compose config</code>	Validate & view the final config

4 depends_on (Start Order)

Use `depends_on` to control the startup order of services.



```
services:
  web:
    image: myapp
    depends_on:
      - db
  db:
    image: mysql:8
```

This doesn't wait for the DB to be "ready", only that it started.

5 Environment Variables & .env

A. In docker-compose.yml

```
services:
  web:
    environment:
      - API_KEY=${API_KEY}
      - DB_HOST=db
```

B. In .env file (same folder)

```
API_KEY=super_secret_key
DB_HOST=db
DB_USER=root
DB_PASS=1234
```

Remember

- .env keeps sensitive info out of the compose file.
- Use `${VAR_NAME}` to read it.

6 Volumes in Compose

Named Volume (managed by Docker)

```
volumes:
  db_data:
```

- ✓ Stored in Docker's area
- ✓ Easy to backup
- ✓ Portable

Bind Mount (Host Path)

```
services:
  web:
    volumes:
      - ./logs:/app/logs
```

- ✓ Map local path to container
- ✓ Good for dev (edit files)

tmpfs (Temporary in-memory)

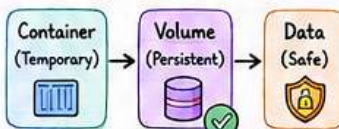
```
services:
  web:
    tmpfs:
      - /tmp
```

- ✓ Stored in RAM
- ✓ Very fast
- ✓ Data lost on container stop



7 Data Persistence

How persistence works?

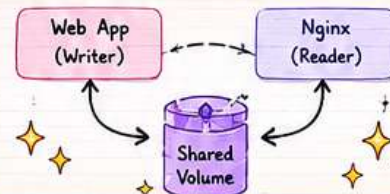


Example

1. Write data → Volume
2. Stop/Remove container
3. Start new container
4. Data is still there! ✓ ♥

8 Volume Sharing Between Containers

Multiple containers can share the SAME volume.



Golden Rule

- ✓ Always use volumes for important data.
- ✓ Never store critical data only inside a container.
- ✓ Compose makes multi-container life EASY!

Common Mistakes

- ✗ Forgetting to mount volumes
- ✗ Using wrong paths in bind mounts
- ✗ Exposing sensitive data in YAML
- ✗ Not using .env for secrets

Pro Tips

- ✓ Use named volumes in production.
- ✓ Use bind mounts for development.
- ✓ Keep .env in .gitignore.
- ✓ Use `docker compose config` to check final YAML.

Quick Command Cheat Sheet

```
docker compose up -d
docker compose down
docker compose build
docker compose logs -f
```

```
docker compose exec web bash
docker compose ps
docker compose config
```


Data is precious! Let's keep it safe with Docker Volumes

DOCKER NOTES

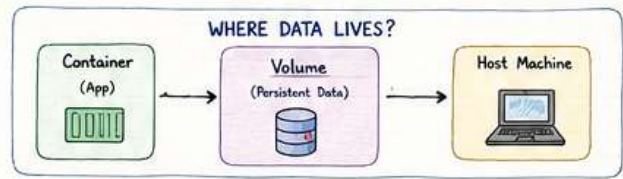


5. VOLUMES & DATA

KEY IDEA

Store data in **VOLUMES** (outside the container) so even if the container is removed, the data stays safe.

Containers are **EPHEMERAL** (temporary). When a container dies, its writable layer is lost. Volumes help us persist and share data safely.



1 VOLUMES vs BIND MOUNTS vs TMPFS

Feature	VOLUMES (Managed by Docker)	BIND MOUNTS (Host Path)	TMPFS (In-Memory)
What is it?	Stored in Docker's managed area on the host.	Map a file or directory on the HOST to the container.	Stored in the HOST memory (RAM), not on disk.
Location	/var/lib/docker/volumes/	Any path on the host system	Inside host memory only
Performance	Good	Good (depends on host disk)	Very Fast (in-memory)
Persistence	Yes (data survives container removal)	Yes (as long as host path exists)	No (data lost on container stop)
Portability	High (easy to move between environments)	Low (depends on host paths)	N/A (temporary)
Use Cases	Databases, logs, backups, production data.	Dev environments, config files, code sharing.	Caching, session data, temp files, sensitive data.

Quick Tip

- Use **VOLUMES** for production data (safer & portable).
- Use **BIND MOUNTS** for development (easy to edit files on host).
- Use **TMPFS** for temporary data that doesn't need to survive.

Remember

Volumes live outside the container lifecycle. Container dies, **DATA SURVIVES!**

2 CREATING & MANAGING VOLUMES

Create a Volume

```
docker volume create mydata
# creates an empty volume
```

Remove a Volume

```
docker volume rm mydata
# must not be in use
```

List Volumes

```
docker volume ls
```

Inspect Volume

```
docker volume inspect mydata
```

Remove Unused Volumes

```
docker volume prune
# removes dangling volumes
```

Use Volume in a Container

-v or --mount

```
docker run -d --name myapp \
-v mydata:/app/data \
nginx
```

mydata = volume name
/app/data = mount point inside container

Check Volume Usage

```
docker system df -v
# shows space used by volumes
```



Common Mistake

Trying to remove a volume that is still used by a container.

Solution:

Stop the container first then remove the volume.

3 DATA PERSISTENCE (KEEP DATA SAFE)

To keep data safe even if a container is removed or recreated.

Example Scenario

You run a database in a container. If you store DB files inside the container, and the container is removed → **DATA LOST!**

Use a volume → **DATA SAFE!**

How Persistence Works



Run DB with Persistent Volume

```
docker run -d --name mydb \
-v dbdata:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=1234 \
mysql:8
```

dbdata = volume name
/var/lib/mysql = where MySQL stores data

Remember

Always mount your important data to a volume!

4 VOLUME SHARING BETWEEN CONTAINERS

Share the same volume between multiple containers.

Example: Two containers sharing the same data volume



Run two containers using the same volume

```
# Writer container
docker run -d --name writer \
-v shared-data:/data \
alpine sh -c "echo Hello > /data/hello.txt && sleep infinity"

# Reader container
docker run -it --name reader \
-v shared-data:/data \
alpine cat /data/hello.txt
```

Output: Hello

They share the same volume!

VOLUME COMMANDS CHEAT SHEET

Action	Command	Example
Create Volume	docker volume create <name>	docker volume create mydata
List Volumes	docker volume ls	docker volume ls
Inspect Volume	docker volume inspect <name>	docker volume inspect mydata
Use in Container	-v <volume>:/path	-v mydata:/app/data
Remove Volume	docker volume rm <name>	docker volume rm mydata
Remove Unused	docker volume prune	docker volume prune

SUMMARY

- ✓ Volumes keep data safe beyond the container lifecycle.
- ✓ Volumes are managed by Docker and are portable.
- ✓ Bind Mounts map host paths (great for dev).
- ✓ tmpfs stores data in memory (super fast but temporary).
- ✓ Share volumes between containers for collaboration.

Golden Rule

If your app creates data, store it in a **VOLUME!**

Future You will thank you!

Think of **VOLUMES** as a **SAFE BOX** that stays even if the container disappears!

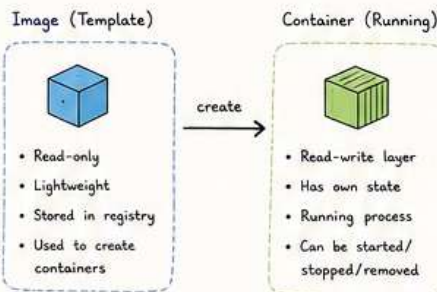


INTERVIEW QUESTIONS

Remember:
Build once,
Run anywhere.

① Q: What is the difference between Docker Image and Docker Container?

A: A Docker Image is a read-only template used to create containers. It is like a blueprint. A Docker Container is a running instance of an image.



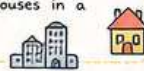
Memory Tip:
Image is like a class, Container is an object.

② Q: What is the difference between Container and Docker Container?

A: Containers share the host OS kernel, so they are lightweight and start very fast. VMs include a full OS (guest OS) with hypervisor, so they are heavier.

Feature	Container	Virtual Machine
Architecture	Share host OS	Full guest OS
Isolation	Process level	OS level
Startup Time	Seconds	Minutes
Size	MBs	GBs
Performance	High	Lower
Use Case	Microservices, Dev/Test	Different OS, Strong isolation

Note:
Containers are like customers in one building. VMs are like houses in a separate neighborhood.



③ Q: What is a Dockerfile and what is it made of?

A: A Dockerfile is a text file that contains instructions to build a Docker Image. It is made of instructions (commands) like FROM, RUN, COPY, CMD, etc.

Common Dockerfile Instructions

FROM	→ Base image
RUN	→ Run commands at build time
COPY	→ Copy files from host
ADD	→ Copy + extra features (url, tar)
CMD	→ Default command when container runs
EXPOSE	→ Document the port
ENV	→ Set environment variables
ARG	→ Build-time variables
WORKDIR	→ Set working directory
USER	→ Run as specific user

Memory Trick:
F R C E A W U
FROM, RUN, COPY, CMD, EXPOSE, ARG, WORKDIR, USER

④ Q: What is the difference between CMD and ENTRYPOINT?

A: CMD provides default command and can be overridden. ENTRYPOINT sets the main command and is usually not overridden.

	CMD	ENTRYPOINT
Purpose	Default command	Main command
Override	Can be overridden	Usually not overridden
Multiple CMD	Only last one used	N/A
Use Case	Provide defaults	Define what container does

Example

Dockerfile	Dockerfile
CMD ["nginx", "-g", "daemon off;"]	ENTRYPOINT ["nginx"]
	CMD ["-g", "daemon off;"]

Tip: Use CMD for options, ENTRYPOINT for the app itself.

⑤ Q: How to run a container in the background (detached mode)?

A: Use the -d or --detach flag.

```
docker run -d --name web nginx
```



Note:
Use docker ps to see running containers (without -a).

```
docker ps  
(running only)
```

⑥ Q: What is the difference between docker stop and docker kill?

A: docker stop sends SIGTERM (polite request) and gives time to clean up. docker kill sends SIGKILL (force) immediately.

Command	docker stop	docker kill
Signal	SIGTERM	SIGKILL
Graceful Shutdown	Yes	No
Use When	Normal stop	Force stop
Default Timeout	10 seconds	Immediate

Warning: Prefer docker stop. Use kill only when a container doesn't stop.

⑦ Q: What is Docker Hub and why do we use it?

A: Docker Hub is a cloud registry (storage for images). We use it to:

- Store and share images
- Pull official or community images
- Push our custom images
- Automate deployments (CI/CD)

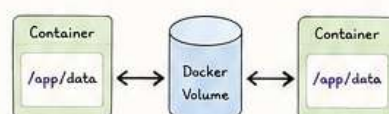


Quick Facts:

- Official images are verified
- Public & Private repositories
- Command: docker push/pull

⑧ Q: What is a Docker Volume and why do we use it?

A: A Docker Volume is a persistent storage managed by Docker. Data in containers is ephemeral (temporary). Volumes help us persist data even if the container is removed.



Benefits

- Data persists after container removal
- Share data between containers
- Better performance than bind mounts
- Easy backup & migration

Memory Tip:
Container is temporary, Volume is permanent.

BONUS CHEAT SHEET

Useful Commands

• docker ps	# List running
• docker ps -a	# List all
• docker images	# List images
• docker run -d --name c1 image	# Run
• docker stop c1	# Stop container
• docker rm c1	# Remove container
• docker rmi image	# Remove image
• docker logs c1	# View logs
• docker exec -it c1 bash	# Access container

Think like Docker:
Build with layers,
Run with containers,
Ship with confidence!



INTERVIEW QUESTIONS

Docker
Advanced
(Senior Level)

1/3

Q1: How do you build the smallest Docker image possible for production?

A:

- Use minimal base image (e.g., alpine).
- Use multi-stage builds.
- Install only what you need.
- Clean up cache and unnecessary files.
- Use .dockerignore.
- Use distroless (no shell) images if possible.

EXAMPLE (Multi-stage)

```
# Build stage
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Production stage
FROM node:18-alpine
WORKDIR /app
COPY --from=build /app/dist ./dist
CMD ["node", "dist/index.js"]
```



MEMORY NOTE

Small Image = Fast Pull +
Less Storage + Less Attack Surface

Q2: How do you handle secrets in Docker without putting them in the image?

A:

- Never put secrets in Dockerfile, image, or environment variables.
- Use Docker Secrets (Swarm) or external secret stores.
- Mount secrets as files at runtime.
- Use .env file (ignored in .gitignore) for local dev only.

Ways to Manage Secrets

Docker Secrets (Swarm)	Built-in secure storage, mounted as files at <code>/run/secrets/<name></code>
Environment File	Use <u>.env</u> for local, NOT in image.
External Secrets	Use tools like HashiCorp Vault, AWS Secrets Manager, Azure Key Vault
Runtime Injection	Inject secrets during deployment (CI/CD)



TIP

Secrets should be injected at runtime, not baked at build time.



Q3: What is Docker Swarm? How is it different from Kubernetes?

A:

Docker Swarm is Docker's native orchestration tool for managing a cluster of Docker nodes. Kubernetes (K8s) is a more advanced orchestration platform with more features.

Swarm vs Kubernetes

Feature	Docker Swarm	Kubernetes
Complexity	Simple & easy	Complex
Setup	Easy (Docker CLI)	Harder
Learning Curve	Low	High
Scalability	Good	Excellent
Features	Basic (built-in)	Very Rich
Ecosystem	Smaller	Huge
Best For	Small/Medium apps	Large/Enterprise apps

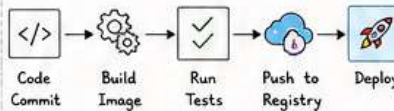
Quick Rule

Swarm = Simplicity | Kubernetes = Power

Q4: How do you use Docker in a CI/CD pipeline? Give an example.

A:

- Build image
- Run tests in container
- Push image to registry
- Deploy to staging/production



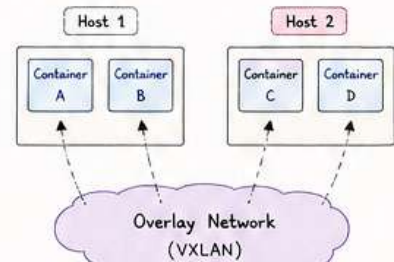
Example (GitHub Actions)

```
name: CI/CD Docker Example
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Build Image
        run: docker build -t myapp:${{ github.sha }} .
      - name: Run Tests
        run: docker run --rm myapp npm test
      - name: Push Image
        run: |
          echo ${{ secrets.DOCKER_PASSWORD }} |
          docker login -u ${{ secrets.DOCKER_USER }} --password-stdin
          docker push myapp:${{ github.sha }}
```

Q5: What is an overlay network in Docker and when is it used?

A:

- Overlay network connects containers across multiple Docker hosts.
- It is used in Docker Swarm (multi-host communication).
- Works at Layer 3 (IP level) using VXLAN encapsulation.



Use Cases

- ✓ Multi-host communication
- ✓ Microservices across nodes
- ✓ Docker Swarm services

2/3

Q6: How do you set resource limits (CPU/Memory) on a container?

A:

- Use `--cpus`, `--memory`, `--memory-swap` flags.
- Helps in performance, stability and cost control.

Examples

```
# Limit CPU to 1.5 cores
docker run -d --name app --cpus="1.5" myapp

# Limit Memory to 512MB
docker run -d --name app --memory="512m" myapp

# Memory + Swap limit
docker run -d --name app --memory="512m" \
  --memory-swap="1g" myapp
```

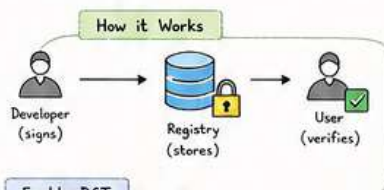
NOTE

`--memory-swap` = total (memory + swap)
Set it equal to memory to disable swap.

Q7: What is Docker Content Trust and how does it improve security?

A:

- Docker Content Trust (DCT) ensures the integrity and authenticity of Docker images.
- It uses digital signatures to verify that an image has not been tampered with.
- Prevents man-in-the-middle attacks and image poisoning.



Enable DCT

```
export DOCKER_CONTENT_TRUST=1
export DOCKER_CONTENT_TRUST_SERVER=https://notary.docker.io

docker push myapp:1.0
# Image is signed and verified on pull
```

Memory Trick



Secure Image + Minimal Image + Limited Resources = Strong & Fast Production System



Q8: How do you troubleshoot a container that is not working properly?

A:

Step-by-step approach:

- 1 Check Container Status
`docker ps -a`
- 2 View Logs
`docker logs <container_id>`
- 3 Inspect Container
`docker inspect <container_id>`
- 4 Execute Inside Container
`docker exec -it <container_id> sh`
- 5 Check Resource Usage
`docker stats <container_id>`
- 6 Check Events
`docker events --since 1h`
- 7 Common Issues
Port conflict, wrong config, insufficient permissions, missing dependencies.

TIP

Logs are your best friend!



KEEP PRACTICING

- Build
- Break
- Fix
- Repeat



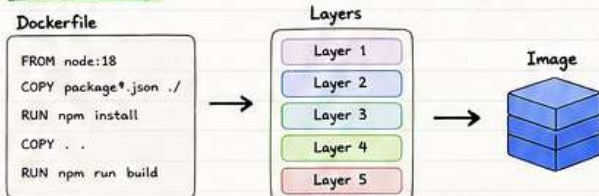
3/3

INTERVIEW QUESTIONS

1) Q: How does Layer Caching work in Docker and how can we optimize build speed?

A: Docker builds images **layer by layer**. Each instruction in the Dockerfile creates a layer. If a layer hasn't changed, Docker **reuses it from cache** instead of rebuilding.

How it works:



How to optimize build speed:

- Order Dockerfile instructions from **least** changing to **most** changing.
- Copy dependency files first (e.g., package.json) and install deps before copying the rest.
- Use `.dockerignore` to exclude unnecessary files.
- Use **multi-stage builds** to reduce final image size.
- Avoid unnecessary RUN commands.

SMART ORDER

Memory Trick
"Stable first,"
then change
the rest!"

Page 1

2) Q: What is Multi-stage Build and when do we use it?

A: Multi-stage build allows you to use multiple FROM statements in a Dockerfile. Each stage can have its own base image and you can copy **artifacts** (files) from one stage to another.

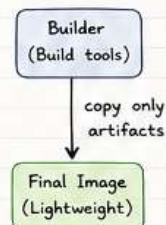
When to use it?

- To build smaller images (only keep what you need).
- To separate build environment from runtime environment.
- To keep secrets and build tools out of the final image.

Example:

```
# Stage 1: Build
FROM node:18 AS builder
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build

# Stage 2: Production
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```



Benefit:



Page 2

3) Q: What is the difference between Volumes and Bind Mounts? When to use each?

Feature	Volumes	Bind Mounts
Managed by Docker	Yes	No (managed by host)
Location	Stored in Docker's storage area	Any path on the host system
Portability	High	Lower (depends on host path)
Performance	Better on Docker Desktop (esp. Mac/Win)	Better on Linux
Use Case	Persistent data, databases, production	Dev environments, config files, source code

When to use?

- Use Volumes for data persistence (survives container removal).
- Use Bind Mounts for development (easier code changes and debugging).

Quick Note
Volumes = Docker manages
Bind Mounts = You manage



Page 3

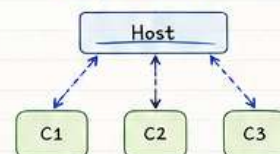
4) Q: What are the types of Docker Networks and when to use each?

Network Type	Description	When to Use
bridge	Default network for containers on the same host	Standalone containers that need to communicate
host	Container shares the host's network stack	High performance applications
none	No networking	Highly secure containers with no network access
overlay	Connects containers across multiple Docker hosts (Swarm mode)	Multi-host applications in Docker Swarm

Tip

bridge = default & simple
host = speed
none = isolation
overlay = scale across hosts

Bridge Network (example)

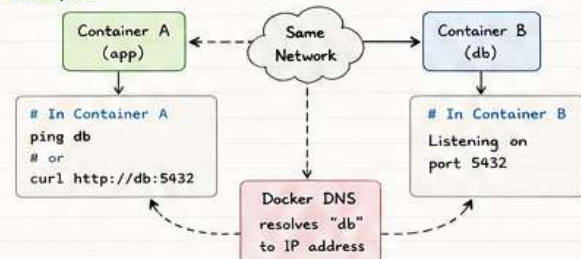


Page 4

5) Q: How does a container communicate with another container in the same network?

A: Containers in the same network can communicate using container name as a hostname. Docker provides an embedded **DNS** (Domain Name System) that resolves container names to their IP addresses.

Example:



Key Point

Use service/container name, not IP address!

Why it's good?

- ✓ IP can change
- ✓ Names are stable
- ✓ Easier to manage

Page 5

6) Q: What is Docker Compose and when do we use it instead of docker run?

A: Docker Compose is a tool for defining and running multi-container Docker applications. You define services in a `docker-compose.yml` file and run everything with a single command.

docker run (Use when)

- Quick test
- Run a single container
- Temporary tasks

VS

Docker Compose (Use when)

- Multi-container apps
- Need networking between services
- Easier to start/stop/manage
- Environments (dev/staging/prod)



7) Q: How do you use environment variables in Docker safely?

- Use `--env` or `--env-file` instead of hardcoding in Dockerfile.
- Use `.env` file with Docker Compose (and add `.env` to `.gitignore`).
- Use **Docker Secrets** for sensitive data in production (Swarm/K8s).



8) Q: What is HEALTHCHECK in Dockerfile and how do we use it?

A: HEALTHCHECK is an instruction to check a container's health periodically. Docker uses it to determine if the container is healthy, unhealthy, or starting.

Example:

```
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
CMD curl -f http://localhost/health || exit 1
```

Why use it?

- ✓ Auto restart unhealthy containers
- ✓ Better monitoring
- ✓ Reliability

Page 6